



Un juego de cartas donde la partida vive en los navegadores de quienes juegan. **No hay servidor de juego.** Nadie tiene la versión buena del estado: o están todos de acuerdo, o no hay partida.

PRIMERO

Qué es y cómo se juega

DESPUÉS

Cómo está hecho

Y EL GRUESO

Cómo se prueba, y con qué

I El juego

Qué es, con qué cartas se juega y cómo entra alguien que pasa por el stand.

Un UNO con temática de desgracias informáticas

Mismo flujo que el juego de cartas de toda la vida: te reparten **7 cartas**, juegas una que **iguale en color, número o símbolo**, y si no puedes, robas. **Gana quien se queda sin cartas.**

Lo que cambia es el disfraz — y algunas mecánicas propias. Aquí no hay un «+2»: hay un *Update de Windows*. No hay «salta»: se te *fue el WiFi*.

Entre **2 y 10 jugadores**, 30 segundos por turno, y se entra escaneando un QR desde el móvil sin instalar nada.

Los cuatro palos sustituyen a los colores. El arte es propio: las cartas están dibujadas en Inkscape y compiladas a un sprite que viaja dentro del HTML, así que están pintadas desde el primer fotograma.



Código



Hardware



Internet



Café

Las de siempre, con otro nombre



Se fue el WiFi
el siguiente
pierde el turno



Ctrl+Z
la ronda cambia
de sentido



Update de Windows
el siguiente
roba y calla



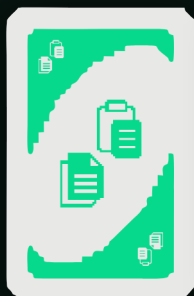
Reinicio de Router
vale sobre todo,
y eliges palo



Pantalla Azul
lo mismo, y el
siguiente roba 4

Un detalle que costó un rato entender jugando: un comodín en el pozo **no dice a qué hay que igualar**. Por eso la carta gira en 3D y aterriza teñida del palo que eligió quien la tiró.

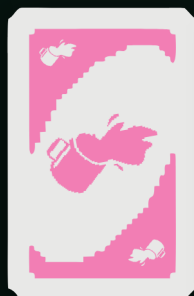
Cuatro mecánicas que el UNO original no tiene



Copiar y Pegar



Apagar y prender



Derrame de Café



Virus Troyano

INTERRUPCIÓN

Copiar y Pegar se juega **fuera de tu turno** si tienes la carta gemela de la del pozo. Corta la ronda y el turno salta a ti.

REINICIO

Apagar y volver a prender: tiras el pozo y pones una carta tuya de base. La partida sigue con ese palo.

BARAJADO

Derrame de Café: todos pasan su mano al siguiente. Tu buena mano deja de ser tuya.

ATAQUE

Virus Troyano: eliges víctima y le regalas dos cartas tuyas. Cuidado: puede dejarte sin mano.

Copiar y Pegar es la que más se nota en el diseño distribuido: es una jugada que llega *fuera de turno* y desde otro nodo, así que dos personas pueden intentar cortar a la vez. Quién gana no lo decide un servidor — lo decide el reloj lógico, y sale igual en las tres pantallas.

Del QR a la primera carta, sin instalar nada

- 1 **Uno hace de anfitrión** y levanta la imagen: `docker run -p 7787:7787 sketox/bug`. Un solo puerto.
- 2 Escribe su nombre, pulsa **Crear sala** y aparece un **código de 4 caracteres** y un **QR**.
- 3 Los demás **escanean el QR** desde su móvil. El enlace ya lleva dentro la sala y el punto de encuentro: solo ponen su nombre.
- 4 Con 2 o más en el lobby, el anfitrión pulsa **¡Empezar!** — y quién abre lo decide la semilla, no él.
- 5 Se juega. **M** abre la **Pantalla Maestra** para el proyector.

REQUISITO DE FERIA

«El público debe poder interactuar de forma inmediata, escaneando un QR, sin instalar aplicaciones».

Por eso el QR transporta **sala + señalización**: quien lo escanea no configura nada. Y si el QR apunta a `localhost`, la propia pantalla lo avisa — solo funcionaría en esa máquina.

DURANTE LA PARTIDA

30 s por turno · recargar la página **no te echa**, vuelves con tu mano · si a alguien se le cae la conexión le guardan el sitio y le saltan el turno.



juego de cartas P2P

Tu nombre

Crear sala

CÓDIGO

Unirse

■ Cómo se juega

· practicar local (hot-seat) ·

La entrada: nombre, crear o unirse, y practicar en local

Cómo se juega

[cerrar x](#)

El objetivo

Quedarte **sin cartas**. Empiezas con 7 y en tu turno tiras una carta que coincida en color o en número con la del pozo. Si no puedes (o no quieres), robas del mazo.



Código Hardware Internet Café

Cuatro palos, del 0 al 9. Sobre el 7 de Internet puedes tirar cualquier otro 7, o cualquier carta de Internet.

Tu turno: 30 segundos

Tienes 30 segundos para jugar. Si no tienes ninguna carta que puedas tirar, el mazo se pone a brillar: robar deja de ser una opción y pasa a ser la opción.

Y no vale escaquearse: no puedes pasar el turno sin haber robado. Si dejas que se acabe el tiempo sin hacer nada, **robas 2 cartas de castigo** y pierdes el turno igualmente.

La regla del "iBug!"



Mesa repartida: siete cartas, mazo, pozo y de quién es el turno



360 px: la anchura desde la que llega casi todo el mundo en la feria

II

Cómo está hecho

Dónde corre el juego de verdad, y qué lo mantiene de acuerdo sin un árbitro.

«Si hay un contenedor encendido, ¿esto no es cliente-servidor?»

El contenedor **reparte el programa y presenta a los jugadores**. El juego **corre entero dentro de cada navegador**.

Cuando alguien abre el enlace, su navegador *se descarga una copia del programa* — como bajarse un PDF. Esa copia se ejecuta en su máquina, con el motor de reglas completo y su propia réplica del estado. El contenedor solo los presenta; a partir de ahí, se aparta.

Es el portero del edificio, no el árbitro de la partida.

La demo que lo demuestra: empezar una partida entre tres y apagar Docker a la mitad. La partida sigue. Solo dejan de poder entrar los nuevos.

	EL CONTENEDOR	TU NAVEGADOR
¿Tiene el motor de reglas?	No	Sí, entero
¿Guarda el estado?	No	Su propia réplica
¿Ve las cartas?	Nunca	Las suyas
¿Decide el turno?	No	El testigo, entre navegadores
¿Sale en la Pantalla Maestra?	No aparece	Sí, como nodo

El WebSocket presenta. El juego va por WebRTC.

ARRANQUE · WEBSOCKET

Encontrarse

Dos navegadores **no pueden encontrarse solos**: no tienen IP pública ni escuchan conexiones entrantes. Alguien tiene que presentarlos. Eso —y solo eso— hace el servidor.

Y hace menos de lo que parece: presenta al que llega con **UN** jugador de la sala. Los demás apretones de manos viajan por la propia malla, dando saltos.

PARTIDA · WEBRTC DATACHANNELS

Jugar

Cada nodo abre un canal con cada otro: **mallá completa**. Las jugadas viajan directas de navegador a navegador, cifradas con DTLS.

Interceptando el WebSocket con un proxy **no se ve ni una carta**: el juego no pasa por ahí.

El bootstrap centralizado es irreducible y lo tienen todos: BitTorrent sus *trackers*, Bitcoin sus *DNS seeds*, IPFS sus nodos de arranque. Lo que se puede elegir es qué pasa después — y aquí, después, el servidor sobra.

Cuatro mecanismos, ningún árbitro

EJE 2 · ORDEN

Relojes de Lamport

Cada jugada lleva su sello lógico. Los eventos concurrentes se desempatan por **peerId**: el mismo orden total en todos, llegue lo que llegue cuando llegue.

EJE 3 · EXCLUSIÓN MUTUA

Testigo de turno

El turno es el recurso crítico. Circula como un testigo con número de secuencia: los anuncios rezagados se descartan solos y nadie puede ceder lo que no tiene.

EJE 3 · CONSISTENCIA

Réplica + huella

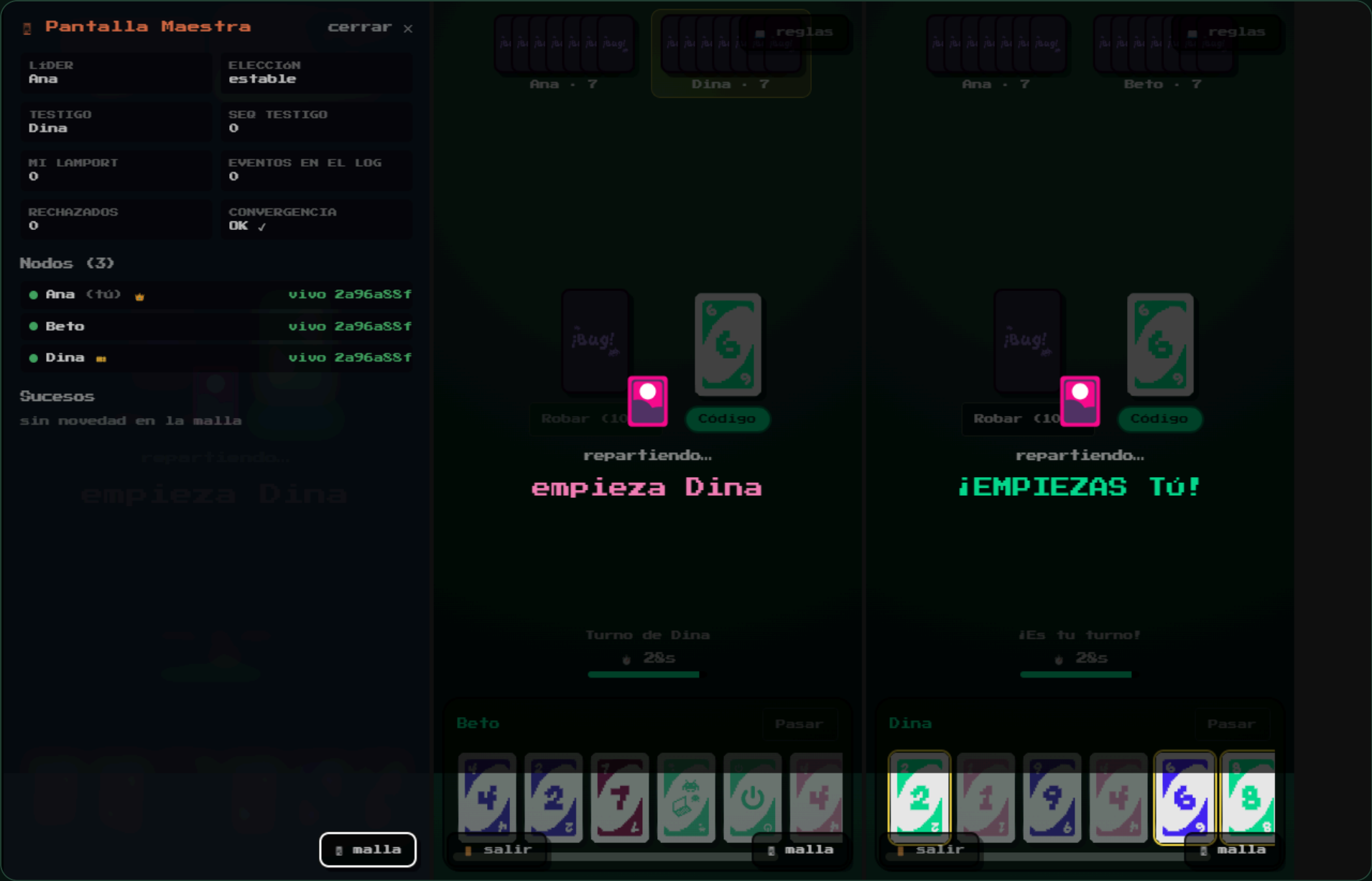
Mismo estado inicial (una semilla) + mismo log = mismo estado. Cada nodo publica el **hash** de su estado en cada latido: la convergencia es verificable a simple vista.

EJE 4 · TOLERANCIA

Latidos + Bully

Silencio de 2,5 s → sospechoso; 6 s → caído. Si cae el coordinador se elige otro con el algoritmo del matón, sin que la mesa se pare.

Y si un nodo **diverge** —le faltó un evento, o le sobra— lo detecta comparando su huella con la del líder, pide la partida entera y **la adopta**. Se repara solo, en vivo, sin que nadie toque nada.



Tecla **M** · quién vive, quién es líder 🏆, dónde está el testigo 🟡 y la huella de cada nodo – la columna que decide si hay partida

III

Cómo se prueba

Qué herramienta para qué pregunta — y qué encontró cada una.

¿Cómo compruebas quién tiene razón si nadie manda?

En una aplicación con backend, comprobar el estado es fácil: se le pregunta a la base de datos. Aquí **no hay a quién preguntar**. Hay tres réplicas y ninguna manda.

Por eso la pregunta central no es *¿está bien construido?* (verificación) ni *¿sirve para lo que se pidió?* (validación), sino una tercera:

¿siguen todos de acuerdo?

Y ADEMÁS

Lo que no existe en Node

`RTCPeerConnection` no existe fuera de un navegador. La malla de verdad **solo** se puede probar en uno.

Y ADEMÁS

Lo que no se repite

Un evento que llega tarde, un nodo que cae a media elección, dos jugadas en el mismo milisegundo. No se reproducen a mano: hay que **provocarlos**.

230 comprobaciones, cada una en la capa que le toca



Encima de todo queda la validación con personas: la feria. Debajo del todo, el análisis estático, que mira incluso el código que nadie llega a ejecutar.

La pirámide es ancha por abajo —190 unitarias sostienen el motor y los algoritmos— pero también **ancha por arriba**, y eso no es doctrina: es lo que dijeron los defectos.

De los 17 encontrados, **11 eran invisibles para una prueba unitaria**. Salieron jugando, en un navegador real, atacando el servidor a mano o montando el propio control de calidad.

Revisa el código entero sin llegar a ejecutarlo

Es **el corrector ortográfico del código**: lo lee entero y señala lo que está mal escrito, sin necesidad de ejecutar nada.

Hace falta porque una prueba solo puede juzgar *lo que llega a ejecutar*. Esto lee **también lo que todavía no usa nadie**, y responde a preguntas que las pruebas ni se plantean: ¿hemos copiado y pegado código? ¿hay alguna parte que ya nadie entiende? ¿estamos probando más que el mes pasado, o menos?

Los cuatro paquetes se revisan como **un solo proyecto**, porque se instalan juntos: así el código repetido entre ellos se ve, en vez de esconderse.



QUALITY GATE

Passed

BUGS

0

fiabilidad A

COBERTURA

89 %

100 % en código nuevo

DUPLICACIÓN

1,3 %

umbral < 3 %

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

 **SonarQube**
community

Projects Issues Rules Quality profiles Quality gates More ▾

🔍 ⓘ Log in

B Bug - juego de carta...
Project

Analysis

 **Overview**

 Issues

 Security hotspots... **Deprecated**

Reporting

 Measures

 Activity

 Quality gate history **New**

Project

<> Code

ⓘ Project information

Bug - juego de cartas P2P descentralizado / Overview

Overview

5.4k Lines of Code · Version 1.7.1 · Last analysis 4 minutes ago

 Quality gate ⓘ

Passed 

New Code

Overall Code

New Code Since August 8, 2026 Started 1 day ago

New issues 0 Required = 0	Accepted issues 0 ⓘ Valid issues that were not fixed	Coverage 100% Required ≥ 80.0% On 2 New Lines to cover.
---	--	---

Duplications
0.0%
Required ≤ 3.0%
On **569** New Lines.

Security Hotspots **Deprecated**
0 **A**

⚠ **Limited security analysis**

SonarQube Community Build does not scan for critical injection vulnerabilities (SQL injection, XSS, and more).

[Explore other editions](#) 

SonarQube Community Build 26.8 · 5 406 líneas · análisis lanzado desde el pipeline

Encontró 23 avisos. Los miramos uno a uno.

1 BUG · ACCESIBILIDAD

El fondo del diálogo de jugada cerraba con ratón y **no con teclado**. Los dos parches evidentes fallaron: un `onKeyDown` en un `div` sin foco no se dispara nunca, y `role="presentation"` cambiaba una queja por dos.

La señal era que el elemento era el equivocado: ahora es un `<dialog>` nativo, que trae hecho `Esc`, foco atrapado y backdrop.

22 AVISOS · TODOS LA MISMA REGLA

`Math.random`. Revisados uno a uno: 16 son partículas de efectos visuales, el resto códigos de sala y semillas **públicos por diseño**.

El único que merecía dudar: `uid()`, que genera el secreto de reconexión. Sus dos ramas reales usan `crypto.0.explorables`.

Y dos incidencias quedaron marcadas como **falso positivo, con la justificación escrita en la herramienta**: el analizador cree que un `<dialog>` no es interactivo. Gestionar hallazgos incluye decidir cuáles no son defectos — lo que no se puede es dejarlos sin mirar.

Cada vez que cambiamos algo, vuelve a probarlo todo

Cada vez que alguien toca una línea, este ayudante **vuelve a pasar las 230 comprobaciones** por su cuenta. Nadie se lo pide.

Hace falta porque las pruebas solo sirven si alguien las corre — y si depende de que uno se acuerde, se acaban corriendo cuando ya se sospecha algo. Así, romper el juego y enterarse tarde **deja de ser posible**: se sabe en minutos.

Y hay algo que solo da esto: probar **en un ordenador que no es el nuestro**.

Varios fallos del semestre eran de los de «en mi equipo funciona».

RESULTADO

SUCCESS

las 7 etapas

DURACIÓN

7,2 min

DISPARO

por commit

sondeo cada minuto

CONFIGURACIÓN

as code

se reconstruye desde cero

1

Dependencias

2

Tipos

3

Pruebas
+ cobertura

4

Build

5

SonarQube

6

Seguridad

7

Validación
distribuida

Ordenadas por **lo que falla más barato primero**: no tiene sentido gastar cinco minutos construyendo Next.js para descubrir después un error de tipos que se detecta en veinte segundos.

 Jenkins / bug-p2p

Status

</> Changes

Git Polling Log

Builds >

Filter

Today

✓ #5 5:06 PM

August 8, 2026

✗ #4 10:13 PM

✗ #3 10:10 PM

✗ #2 10:08 PM

✗ #1 9:52 PM

✓ bug-p2p

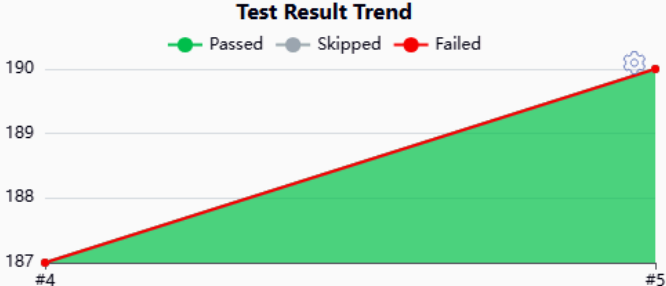
Monorepo de Bug. Dispara con cada commit de la rama principal (sondeo del repo montado en /repo-src).

Last Successful Artifacts

lcov.info	41.41 KiB	view
distribuida-2026-08-10T17-13-34.json	6.97 KiB	view
distribuida-ultimo.json	6.97 KiB	view
seguridad-2026-08-10T17-13-05.json	7.07 KiB	view
seguridad-ultimo.json	7.07 KiB	view

Latest Test Result (no failures)

Test Result Trend



Permalinks

- Last build (#5), 9 min 11 sec ago
- Last stable build (#5), 9 min 11 sec ago
- Last successful build (#5), 9 min 11 sec ago
- Last failed build (#4), 1 day 19 hr ago
- Last unsuccessful build (#4), 1 day 19 hr ago
- Last completed build (#5), 9 min 11 sec ago

REST API

Jenkins 2.568.2

El historial cuenta la verdad: #1 a #4 en rojo son el laboratorio que no arrancaba; #5 es el primer verde, disparado por un commit – 190 pruebas sin fallos y los artefactos archivados

Estaba escrito desde hacía semanas. No había arrancado ni una vez.

- 1 **Jenkins no arrancaba.** El Job DSL dinámico (`scmGit { remotes }`) ya no existe en las versiones de hoy: se llevaba por delante a Configuration-as-Code y el contenedor moría en `Exited (5)`.
- 2 **La corrección no llegaba.** `casc.yaml` vivía dentro de `/var/jenkins_home`, que es un volumen: ahí manda el volumen, no la imagen. Reconstruir no cambiaba nada.
- 3 **El checkout moría.** El plugin de Git no clona rutas locales sin habilitarlo explícitamente.
- 4 **Y moría otra vez, un paso más adentro.** Git rechazaba el repo montado por pertenecer a otro usuario. El mismo error del punto 2 con otro disfraz: `--global` escribe en `$HOME...` que es el volumen.

Y uno silencioso, el peor: el token de Sonar nunca entraba en el contenedor, así que el pipeline habría saltado el análisis en cada build **con el mismo amarillo que si el laboratorio estuviera apagado**. Un fallo disfrazado de comportamiento tolerado no lo investiga nadie.

Abre el juego, mueve cartas y comprueba que todo cuadra

Un robot que **juega como jugaría una persona**: escribe su nombre, pulsa «crear sala», mira sus cartas y tira una.

Y hace algo que una persona no puede: abrir **tres jugadores a la vez** y comprobar, después de cada jugada, que los tres están viendo exactamente la misma partida.

Se eligió Cypress y no Selenium porque **corre dentro del navegador**, al lado del juego: puede mirar directamente lo que el juego tiene en la cabeza. Selenium lo maneja desde fuera, y todo lo que quiera leer tiene que pasar por un intermediario.

MENU.CY.TS

8

entrada, QR, validación del enlace, móvil de 360 px

PARTIDA-LOCAL.CY.TS

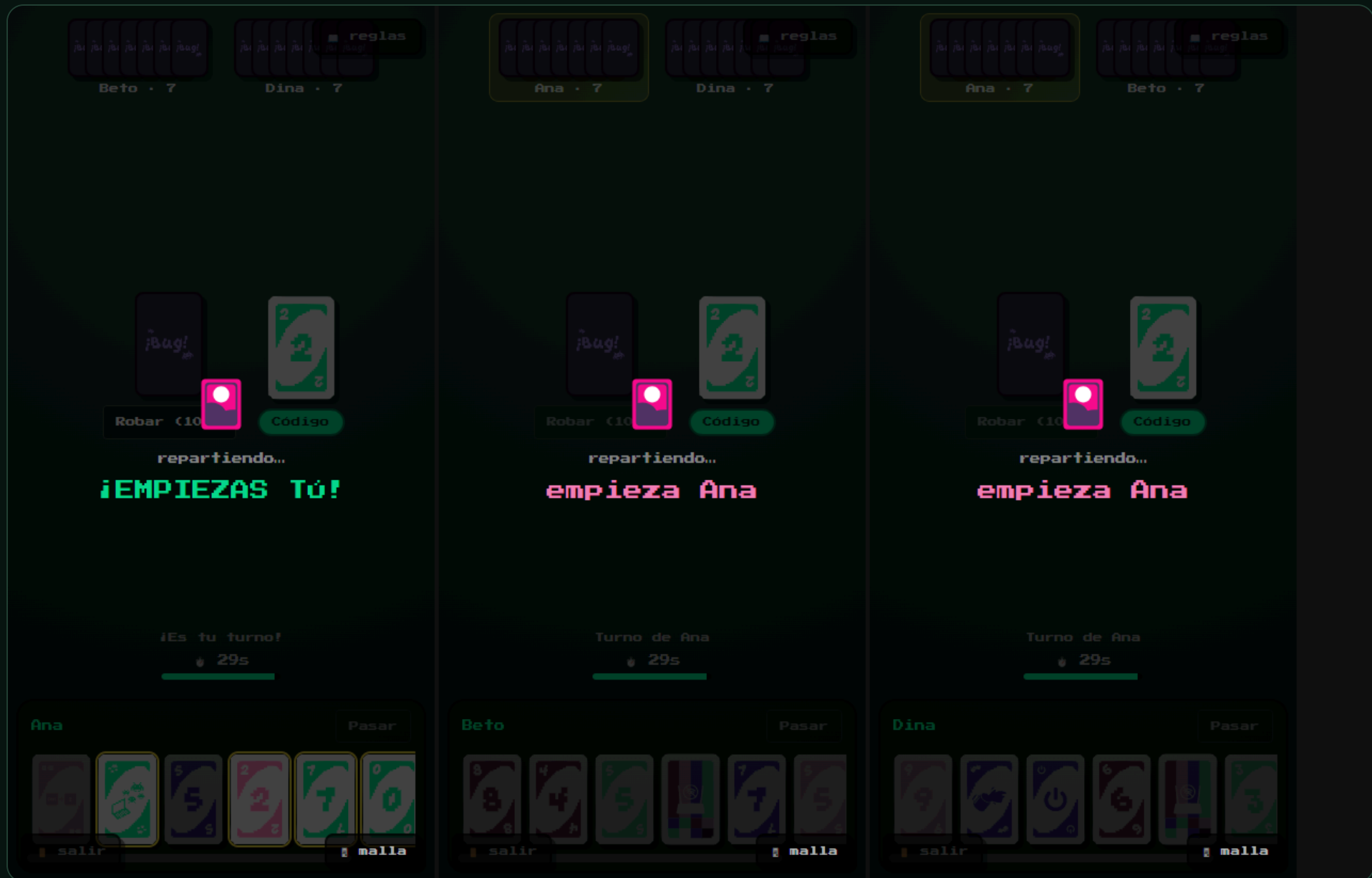
6

reparto, robar, no pasar sin robar, rotación del turno

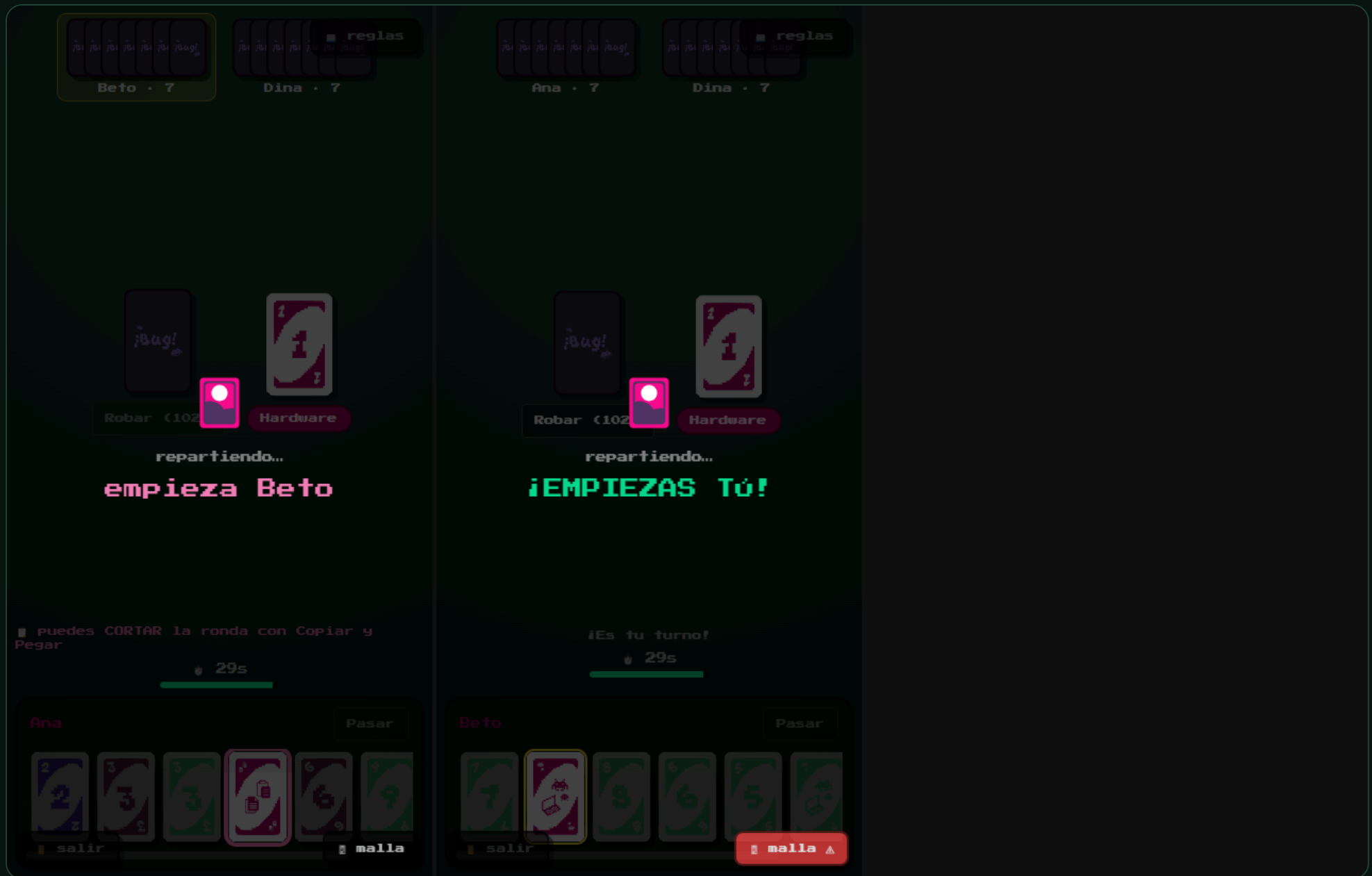
DISTRIBUIDO/MALLA.CY.TS

5

Tres nodos con WebRTC real: lobby, convergencia, jugada compartida, Pantalla Maestra y caída



Ana, Beto y Dina · tres contextos de navegación con su propia `RTCPeerConnection` · la prueba compara sus tres huellas de estado



Se quita el tercer nodo de golpe · los dos que quedan siguen convergiendo entre ellos

Hacer trampas de verdad, para ver si el juego aguanta

Nos sentamos **en medio de la conversación** entre jugadores, cambiamos los mensajes y los reenviamos. Once trampas distintas.

Hacía falta porque todo lo demás usa el juego *como está previsto*, y quien quiere colarse no hace eso.

Las once quedaron **escritas como prueba automática**. Un agujero que solo está en la cabeza de quien lo encontró vuelve a abrirse en dos semanas.

LA TRAMPA QUE PROBAMOS	ANTES	AHORA
Hacerse pasar por otro jugador	×	✓
Echar a alguien de su propia partida	×	✓
Tumbar el servidor con un mensaje raro	×	✓
Meterse en una partida ajena	×	✓
Saturarlo abriendo miles de conexiones	×	✓
Mandarle un mensaje de 32 MB	×	✓
Otras cinco, que ya aguantaba	✓	✓

× se colaba · ✓ la bloquea · **6 agujeros tapados**

Un mensaje de dos líneas tumbaba el servidor de toda la feria

1 · QUÉ MANDAMOS

Un mensaje normal del juego, pero con **un número donde tenía que ir una lista**.

Desde la consola del navegador. Lo puede hacer cualquiera que esté jugando.

2 · QUÉ PASABA

El servidor no sabía qué hacer con eso y **se apagaba entero**.

Nadie más podía entrar a ninguna partida. En una feria, se acabó la demo.

3 · CÓMO SE ARREGLÓ

Ahora **revisa todo lo que le llega** antes de tocarlo, y lo raro lo tira a la basura.

Y una red de seguridad debajo, por si se nos escapa otro que no imaginamos.

Las partidas en curso **habrían seguido jugándose igual** — las cartas no pasan por el servidor. Pero nadie nuevo habría podido entrar.

Provocamos los desastres que en la vida real casi nunca pasan

Jugamos **miles de partidas entre jugadores imaginarios** y, a propósito, hacemos que la red se porte mal. Después comprobamos que todos acaban viendo lo mismo.

RETRASAMOS

Una jugada tarda en llegar y aparece **fuera de orden**.

DUPLICAMOS

La misma jugada llega **dos veces**.

DESCONECTAMOS

Un jugador **desaparece** a media partida.

CORROMPEMOS

A alguien le **cambiamos las cartas** a mano.

RESULTADO

7 de 7

propiedades verificadas

200 VECES

nos pasamos el turno de mano y **ni una vez** lo tuvieron dos a la vez

EL CORRUPTO

se da cuenta solo, **pide la partida entera** y vuelve a la mesa

Hubo que escribirlo nosotros: ninguna herramienta que se pueda comprar sabe qué es «el turno de Bug» ni cuándo dos jugadores están de acuerdo.

Casi ninguno salió de las pruebas automáticas del principio

JUGANDO DE VERDAD

La sala no se creaba

Pulsabas «Crear sala» y no pasaba nada. Las pruebas daban todo verde: el fallo estaba en cómo el navegador monta la pantalla, no en las reglas del juego.

JUGANDO DE VERDAD

Un jugador se quedaba fuera

Veía otra partida distinta a la de los demás y no podía tirar ninguna carta. Le había faltado una jugada por el camino.

JUGANDO DE VERDAD

No cabía en el móvil

La pantalla de entrada se salía por los lados en un teléfono — que es por donde entra casi todo el mundo en la feria.



Por eso no basta con probar el código por dentro: hay que **abrir el juego y jugarlo**, con tres jugadores de verdad y en un móvil de verdad.

Lo que queda por hacer

LO PRIMERO

Probarlo entre dos casas

Funciona en la misma WiFi y con un túnel. Falta la prueba con dos redes distintas de verdad — el código ya está preparado.

PARA EL STAND

Diez móviles a la vez

Está medido con diez jugadores simulados. Falta hacerlo con diez teléfonos en la mano.

A FUTURO

Probar el juego en cada cambio

Las pruebas del navegador se lanzan a mano; el resto ya se lanza solo.

Saber dónde **no** se ha mirado todavía es parte del trabajo. Lo que no está medido, se dice.



Un juego que se sostiene solo,
y 230 comprobaciones que lo vigilan

CALIDAD DEL CÓDIGO

Passed

0 errores · 89 % revisado

SE COMPRUEBA SOLO

7/7

en cada cambio

SE JUEGA SOLO

22/22

3 jugadores a la vez

TRAMPAS · DESASTRES

11 · 7

bloqueadas · superados

Gracias.